Acompáñame · Decisiones técnicas y proceso

21 oct 2026 · Andrea Vallina González

1. Contexto

Acompáñame es un marketplace que conecta a familias que necesitan cuidados puntuales para una persona mayor o dependiente con cuidadores profesionales. Es el proyecto final del bootcamp de Desarrollo Web Full Stack (850 h) de Factoría F5, realizado en solitario.

El proyecto se compone de dos repositorios independientes: un backend (API REST con Spring Boot) y un frontend (aplicación Vue 3). La base de datos MySQL corre en Docker.

Aquí explico por qué tomé cada decisión, no solo lo que hice: qué opciones tenía, cuál elegí y qué he dejado para más adelante. Va junto a los README, las wikis, los diagramas y el tablero de JIRA.

En la entrega (12 y 13 de octubre) están el registro y el login, la búsqueda de cuidadores, las solicitudes con sus estados, las valoraciones y un pago de prueba. Después de la entrega he seguido mejorando el código (sin cambiar lo entregado, que queda marcado con la etiqueta `delivery-mvp-2026-10-12`): valoraciones privadas, tests de entidad, refactorizar el CORS y los nombres de carpetas, las páginas legales y, como novedad principal, el **panel de administración** (sección 5). Lo demás (chat, geolocalización, videollamadas, recordatorios, informes en PDF y pago real) queda para una segunda fase (sección 10).

2. Arquitectura y stack

El backend sigue el patrón que vimos en clase: Controller, Service, Repository y Entity. Cada capa tiene una sola tarea: el controller recibe la petición, el service tiene las reglas del negocio y el repository habla con la base de datos. El código está agrupado por temas (usuario, solicitud, pago, admin...) para tener junto lo que se toca a la vez. Los nombres de las carpetas siguen una sola convención en inglés (`caregiverprofile`, `servicerequest`, `review`, `admin`...).

Decisión	Por qué
Java 21 y Spring Boot	Es lo que trabajamos en el bootcamp, y ya trae seguridad y acceso a base de datos. Me quedo en Java 21 porque Lombok da errores de compilación con versiones más nuevas sin configuración extra.
DTOs y Mappers	La API nunca devuelve las entidades tal cual. Así no se escapa la contraseña y el cliente solo puede enviar los campos que yo permito.
Interfaz genérica `InterfaceGenericService`	Varias entidades tienen el mismo CRUD; así escribo el contrato una sola vez.
Inyección por constructor	Es más limpia y facilita los tests con Mockito.
Excepciones propias y `GlobalExceptionHandler`	Cuando algo falla, la API responde con un código claro (401, 403, 404, 409) y un mensaje sin datos internos.
Lombok	Evita escribir a mano getters, setters y constructores.
MySQL en Docker Compose	Se levanta con un solo comando y funciona igual en cualquier ordenador.

Decisión	Por qué
`ddl-auto=update`	No se borran los datos cada vez que reinicio la aplicación.
Vue 3 con `<script setup>`, Vite y Pinia	Menos código para escribir y arranque rápido.
CSS puro con BEM y variables	Control total del diseño sin añadir otra librería (ver sección 6).

Las credenciales de la base de datos no están escritas en el código: se leen de variables de entorno (`DB_HOST`, `DB_USER`, `DB_PASSWORD`…) desde un archivo `.env` que no se sube a Git. En el repositorio hay un `.env.example` como plantilla. El origen permitido por CORS se lee igual, de `CORS_ORIGINS`, y la cuenta del administrador, de `ADMIN_EMAIL` y `ADMIN_PASSWORD`. Spring no lee el archivo `.env` por sí solo: las variables hay que tenerlas definidas en el entorno al arrancar.

3. Base de datos y modelo de datos

Seis entidades: Usuario, Role, PerfilCuidador, Solicitud, Valoracion y Pago.

Relación	Decisión
Usuario ↔ Role	@ManyToMany con los roles en tabla propia, no en un enum.
Usuario → PerfilCuidador	@OneToOne: solo los cuidadores tienen perfil profesional.
Solicitud → Usuario y PerfilCuidador	@ManyToOne en ambos: una familia hace muchas solicitudes y un cuidador recibe muchas.
Valoración → Solicitud	@OneToOne: se valora un servicio concreto y solo una vez, cuando está completado.
Pago → Solicitud	@OneToOne con estados PENDIENTE, COMPLETADO y CANCELADO.
Usuario.activo	Campo booleano (por defecto verdadero) añadido para el panel de administración: bloquear a un usuario es poner `activo = false`, sin borrar su historial.

Los roles los guardo en una tabla y no en un enum. Un enum habría valido porque cada usuario tiene un solo rol, pero la tabla es lo habitual en Spring Security, permite que un usuario tenga varios roles y se pueden añadir roles nuevos sin tocar el código. Eso se comprobó al añadir el rol ADMIN: no hubo que cambiar el modelo. Los roles FAMILIA, CUIDADOR y ADMIN se crean solos al arrancar (`RoleInitializer`).

Usuario es una sola tabla. Pensé en separar los datos de login de los datos personales, pero para este proyecto solo lo complicaba. Si la app creciera, lo haría.

Una solicitud puede estar PENDIENTE, ACEPTADA, RECHAZADA o COMPLETADA. El estado se cambia con `PATCH /solicitudes/{id}/estado`. Uso PATCH y no PUT porque solo cambia un dato. Por la misma razón, cambiar la contraseña y el contacto del usuario son PATCH, y también bloquear o desbloquear a un usuario (`PATCH /admin/usuarios/{id}/activo`).

4. Seguridad

Decisión	Por qué
Basic Auth (email y contraseña)	Es lo más sencillo con Spring Security y es lo que dominaba. Sirve para demostrar login y permisos. JWT queda como mejora.
BCrypt	La contraseña se guarda cifrada y nunca se devuelve.
`JpaUserDetailsService` propio	Spring comprueba el login contra los usuarios reales de la base de datos. También es donde se rechaza a los usuarios bloqueados.
`hasAuthority` en vez de `hasRole`	`hasRole` añade solo la palabra ROLE_ delante. Con `hasAuthority` se compara el nombre exacto que hay en la base de datos.
Permisos por rol	Solo un cuidador puede crear, editar y borrar perfiles; solo una familia puede hacerlo con solicitudes; solo un administrador entra en `/api/v1/admin/**`.
Cada usuario solo toca lo suyo	Aunque alguien conozca el id de la solicitud de otra persona, la API le responde «acceso denegado». Esta comprobación está en el service. (Es lo que se llama IDOR.)
Valoraciones filtradas en el servidor	Antes GET /valoraciones devolvía todas y el frontend filtraba, así que cualquiera podía ver valoraciones ajenas llamando a la API. Ahora el service solo devuelve las de la familia o el cuidador que ha iniciado sesión, y ver una por id solo se permite a los dos participantes de esa solicitud (si no, 403).
CORS desde una propiedad (`cors.allowed-origins`)	La API solo acepta peticiones desde mi frontend. La dirección se cambia con CORS_ORIGINS al desplegar, sin tocar el código.
Pocas rutas públicas	Solo registro, lista de roles, login, ver cuidadores y las valoraciones públicas de un cuidador. Todo lo demás pide estar identificado.

Con Basic Auth el frontend guarda el email y la contraseña en sessionStorage para enviarlos en cada petición. Para una demo vale, y con JWT se mejoraría. También hay un caso límite pendiente de revisar: llamar a `/login` sin credenciales, que al ser una ruta pública podría dar un error 500 en vez de 401.

5. Panel de administración

Es la última pieza del proyecto: una pantalla (`/admin`) para que alguien de la plataforma pueda ver cómo va y retirar lo que no debe estar. La idea que lo guía es que **el administrador observa y modera, no participa**: no crea solicitudes, no paga, no valora.

Decisión	Por qué
Qué puede hacer	Ver estadísticas y avisos, listar usuarios, ver las últimas valoraciones, los cuidadores mejor valorados y las solicitudes filtradas por estado. Solo dos acciones que cambian datos: eliminar una valoración y bloquear o desbloquear a un usuario.
Rol ADMIN en la misma tabla de roles	No hizo falta tocar el modelo. Todas las rutas `/api/v1/admin/**` exigen la autoridad ADMIN con `hasAuthority`.
Bloquear en vez de borrar	Un usuario con `activo = false` conserva sus solicitudes y valoraciones, y se puede recuperar. Borrarlo rompería el historial de otras personas.

Decisión	Por qué
Bloqueo con efecto inmediato	Como Basic Auth comprueba el usuario en cada petición, el bloqueado recibe 401 en la siguiente llamada, igual que con una contraseña incorrecta. No hay que invalidar ninguna sesión.
El administrador no puede bloquearse a sí mismo	El service lo comprueba y responde 403. Evita dejar la plataforma sin nadie que la administre.
Confirmación antes de eliminar	La vista pide «¿Eliminar esta valoración?» antes de borrar, porque la acción no se deshace.
Cuenta creada por variables de entorno	`AdminInitializer` crea al administrador al arrancar con ADMIN_EMAIL y ADMIN_PASSWORD. Así no hay contraseña escrita en el código ni un registro público para ser administrador.
Endpoints	GET `/admin/estadisticas`, `/avisos`, `/cuidadores/mejor-valorados`, `/valoraciones/ultimas`, `/usuarios` y `/solicitudes?estado=`; DELETE `/admin/valoraciones/{id}` (204) y PATCH `/admin/usuarios/{id}/activo`.

6. Frontend

Decisión	Por qué
Patrón Repository	Hay una clase por cada recurso (SolicitudRepository, AdminRepository, etc.) que hace las llamadas al backend. Las pantallas no saben nada de fetch ni de la autenticación, y en los tests basta con simular el repositorio.
Pinia con sessionStorage	Guarda quién ha iniciado sesión. Si recargo la página sigue ahí, y se borra al cerrar la pestaña.
Vue Router	Una ruta por pantalla, con acceso según haya sesión y según el rol (ahora también ADMIN).
Una sola NavBar	El menú cambia según seas Familia, Cuidador, administrador o visitante, sin repetirlo en cada pantalla.
Componentes reutilizables	SolicitudCard se usa en dos pantallas; el panel de administración se divide en componentes pequeños (estadísticas, avisos, tablas) dentro de `components/admin`.
BEM y variables CSS	Los colores y espacios están en un solo sitio; para cambiar la paleta toco un archivo.
Responsive a 768 px	Un solo punto de cambio entre móvil y escritorio, para mantener el CSS sencillo. El panel de administración también se adapta a móvil.
Pantallas vacías y mensajes de error	La app nunca se queda en blanco si algo falla o no hay datos.
Validar formularios	Compruebo los campos obligatorios antes de enviar. El backend valida también; la del frontend es para avisar rápido al usuario.
Modal de valoración y páginas legales	Se valora desde el historial, con una ventana que evita cambiar de pantalla. Las páginas legales cumplen con lo que se espera de una web real.

El pago es de prueba: la pantalla funciona completa y llama al backend con `PATCH /pagos/{id}/pagar` para marcar el pago como completado, pero los datos de la tarjeta son de prueba y no se cobra de

verdad. Lo primero que haría después de la entrega es conectarlo con una pasarela de pago real, por ejemplo Stripe en modo test.

7. Pruebas

Qué pruebo	Con qué	Para qué
Backend: servicios	JUnit 5 y Mockito	Probar la lógica de cada service sin base de datos, simulando los repositorios.
Backend: controladores	MockMvc y @WebMvcTest	Comprobar que cada endpoint devuelve el código y el JSON esperados.
Backend: entidades	JUnit 5 y Hamcrest	Comprobar constructores, getters y setters de las entidades, sin Spring ni base de datos.
Backend: integración	Testcontainers	Probar con una MySQL de verdad (en Docker) los repositorios y la seguridad, incluido el acceso al panel de administración.
Frontend	Vitest, Vue Test Utils y Playwright	470 tests unitarios en 46 archivos, simulando los repositorios y sin necesitar el backend, más 9 tests e2e que recorren la aplicación completa en un navegador.

Uso Testcontainers y no una base de datos en memoria porque una base distinta de la real puede dar el test por bueno y luego fallar en MySQL. Tarda más y necesita Docker encendido, pero prueba lo que de verdad se usa.

Me centré en los casos límite: validaciones, errores de carga y que un usuario no pueda ver o tocar datos de otro. Los tests de casos de error y de seguridad (400, 401, 403, 404 y 409) me sirvieron para encontrar fallos reales: un pago inexistente daba 500 en vez de 404, y cualquier usuario podía editar o borrar la valoración de otra persona.

Para el panel de administración probé que un usuario sin el rol ADMIN recibe 403, que el bloqueado recibe 401, que el administrador no puede bloquearse a sí mismo y que eliminar una valoración responde 204. El backend tiene ahora **436 tests** (antes 298), todos en verde, con una cobertura JaCoCo del 96 % en líneas y el 95 % en ramas. El frontend tiene **470 tests unitarios** (60 de ellos del panel) y 9 e2e, con un 98 % de cobertura.

8. Diseño y gestión del proyecto

Primero hice bocetos, luego wireframes, mockups (móvil, tablet y escritorio) y un prototipo en Lovable. Así pude decidir pantallas, flujos y colores antes de programar, y después construir la app comparando con una referencia. Del prototipo cogí la paleta verde con acento coral, la letra Nunito y la barra de navegación. En Figma están los wireframes y mockups.

Me puse una regla: si una pantalla no está en el diseño, no se hace. Por eso quité la pantalla de editar perfil del cuidador, que había añadido yo sin estar en el prototipo, y al final la recuperé para la entrega, ya con foto de perfil. El panel de administración se añadió después de la entrega y no estaba en el prototipo: lo construí con la misma paleta, tipografía y componentes, y lo documenté con capturas en escritorio y móvil.

Para organizar el trabajo usé Scrum con JIRA (proyecto ACOM).

Elemento	Cantidad
Épicas	9
Historias de usuario (criterios en Gherkin)	19
Tareas técnicas	120
Sprints (agosto a octubre de 2026)	6

Las épicas son: Autenticación y Seguridad, Gestión de Cuidadores, Solicitudes de Servicio, Valoraciones, Pagos, Calidad y pruebas, Diseño y UX, Infraestructura y entrega, y Panel de administración. Se eligió Gherkin (dado, cuando, entonces) porque convierte cada criterio de aceptación en algo directamente comprobable con un test.

En Git hice un commit por cada cambio, con los mensajes en inglés y el número de tarea al final, por ejemplo (ACOM-148). Al principio trabajé en main porque el proyecto es individual. Cuando el profesor me pidió una rama `dev`, la creé y desde ahí sigo: cada mejora va en su rama (`feature/...`, `fix/...`, `docs/...`), se une a dev y, cuando está terminada, documentada y en verde, a main. Uso uniones `--ff-only` para que el historial sea lineal y una etiqueta (`delivery-mvp-2026-10-12`) para proteger la versión entregada.

9. Problemas encontrados y cómo se resolvieron

Problema	Qué pasaba	Cómo lo resolví
Los roles estaban escritos a mano en el registro	En el formulario puse los números de los roles. Si la base de datos cambiaba, el registro dejaba de funcionar.	El formulario pide los roles al backend (GET /roles) y los roles se crean solos al arrancar.
Los permisos por rol no se comprobaban como esperaba	Spring añade él solo la palabra ROLE_ delante del nombre del rol y mis roles no la llevaban.	Uso hasAuthority, que comprueba el nombre tal cual está en la base de datos.
Seguridad con JWT sin haberlo visto en clase	Mientras desarrollaba, el profesor aún no había explicado JWT.	Empecé con Basic Auth, que sí dominábamos, y dejé JWT como mejora futura.
Cambios de última hora en la base de datos	A mitad de proyecto añadí el campo «vehículo propio» y la entidad Pago.	Actualicé la base de datos y los diagramas. Aprendí a cerrar el diseño de datos antes de programar.
Lovable no aplicaba todos los cambios	Le mandaba una lista de mejoras y alguna se quedaba sin hacer.	Revisaba el resultado punto por punto y volvía a pedir lo que faltaba.
Estilos que no se aplicaban	Escribí &--modificador como en Sass dentro de un CSS normal. El navegador lo ignora sin avisar.	Escribí cada selector completo (.btn--primary).
Una pantalla de más	Construí la edición de perfil sin que estuviera en el prototipo.	La quité y más tarde la recuperé para la entrega.

Problema	Qué pasaba	Cómo lo resolví
La app no se parecía al prototipo	Había construido desde los mockups sin comparar con Lovable.	Reescribí la barra de navegación, puse el fondo verde y cambié los iconos por SVG.
Los tests de integración fallaban	Necesitan Docker encendido para levantar la base de datos de pruebas.	Lo dejé escrito en el README: encender Docker antes de ./mvnw test.
Las valoraciones de otros usuarios se podían ver	GET /valoraciones devolvía todas y el frontend filtraba.	Filtro en el servidor, compruebo al ver una por id y añado tests para cada caso.
Errores de Lombok al compilar	Maven usó Java 25 y Lombok no generaba getters ni constructores.	Volví a Java 21 (JAVA_HOME apuntando a JDK 21) y dejé el pom.xml en 21.
La aplicación no arrancaba con el administrador	Con todos los tests en verde, al arrancar de verdad con ADMIN_EMAIL y ADMIN_PASSWORD fallaba con `LazyInitializationException` al crear la cuenta del administrador: los roles se cargan de forma perezosa y la sesión ya estaba cerrada.	Hice el inicializador `@Transactional`, para que la sesión siga abierta mientras trabaja con los roles. Además añadí una prueba manual de arranque a mi lista antes de entregar.
Spring no leía el archivo .env	Definí las variables del administrador en .env y la aplicación no las veía.	Spring no lee .env por sí solo: las variables deben estar en el entorno al arrancar. Quedó explicado en el README y en la wiki.
Git marcaba archivos como modificados sin cambios	Era solo una diferencia de saltos de línea entre Windows y Linux.	No hizo falta hacer nada; añado los archivos de uno en uno, nunca con git add .

Lo que más he aprendido: comparar la interfaz con el prototipo desde el principio y no al final; no añadir nada que no esté en el diseño; que no basta con ocultar datos en el frontend, porque la privacidad se comprueba siempre en el servidor; y que **los tests en verde no sustituyen a arrancar la aplicación de verdad**.

10. Limitaciones conocidas, mejoras y fase 2

Limitaciones conocidas:

- Autenticación Basic Auth con credenciales en sessionStorage.
- Pago de prueba: la tarjeta es de prueba y no se cobra nada.
- El valor por defecto de desarrollo de la base de datos sigue en `docker-compose.yml`.
- Algunas pantallas no coinciden al detalle con el prototipo de Lovable, y hay colores escritos directamente en el CSS de algunas vistas en vez de usar las variables de la paleta.
- Un único punto de corte responsive (768 px).
- Sin límite de intentos de login y `/login` sin credenciales por revisar.

- Panel de administración: solo hay una cuenta de administrador, creada desde variables de entorno, y no se registran las acciones de moderación (quién bloqueó a quién).
- Sin despliegue: todo corre en local.

Lo que haría a continuación, por orden:

1. Migrar de Basic Auth a JWT, en una rama aparte y solo si todos los tests siguen en verde.
2. Conectar el checkout con una pasarela de pago real, por ejemplo Stripe en modo test.
3. Guardar las fotos de perfil como archivos externos, en lugar de dentro de la base de datos.
4. Registrar las acciones del administrador.
5. Desplegar frontend y backend, definiendo las variables de entorno en el servidor.

Para una segunda fase: chat en tiempo real, geolocalización de cuidadores, videollamadas, recordatorios y notificaciones, informes en PDF y pago real.

Se separó desde el principio lo que entra en la entrega de lo que queda para esta fase, para terminar un producto completo y no uno a medias.